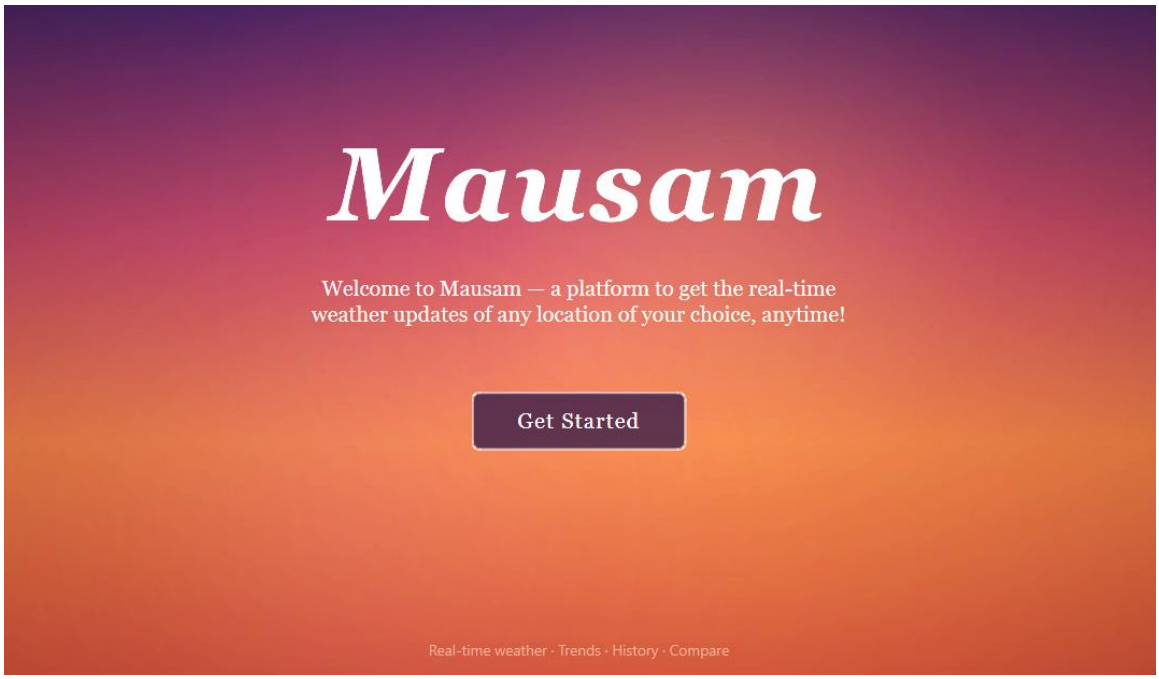


Project Mausam

WEATHER MONITORING & ANALYSIS SYSTEM (API-BASED)

Team Members:

- **Priyasmita G (Leader) – 2025249607**
  - **Role:** Project Overseer & Report Writer (with GUI Support)

- **G. Sai Satwik – 2025030281**
  - **Role:** Lead GUI Designer

- **G. Tejaswini – 2025016703**
  - **Role:** Connector (Backend - GUI Linking with API Integration)

- **Sai Prasad – 2025319186**
  - **Role:** Backend Developer & Version Control Head

Overview:

The **Weather Telemetry System** is a Python-based application developed for the **GITAM PPS-II short-term project**.

The **Weather Monitoring & Analysis System** is a Python-based application designed to fetch and analyze real-time weather data using a public API, **Open-Meteo API**. Users can input a city name to view temperature, humidity, pressure, and conditions in a clear format. The system stores daily reports, compares weather between cities, and identifies the hottest or coldest days. Advanced features include a GUI for interactive use and graphical trend analysis with charts. By combining API requests, JSON parsing, file handling, and exception management, the project delivers a practical tool for monitoring weather patterns and gaining meaningful insights from stored data.

This project demonstrates practical implementation of:

- API-based data acquisition
- Data storage and processing
- Weather trend analysis
- GUI development
- Data visualization

Objectives of the Project:

Primary Goal: Develop a Python-based application that fetches, displays, stores, and analyzes real-time weather data for cities using a public weather API.

Core Objectives

1. **Real-Time Data Retrieval** – Fetch live weather data (temperature, humidity, pressure, weather condition) for any user-specified city via a public weather API.
2. **City Comparison** – Allow users to compare weather parameters of two different cities side by side.
3. **Data Storage** – Store daily weather reports persistently in a file for future reference and analysis.
4. **Trend Analysis** – Identify and display weather trends such as the hottest/coldest day from stored historical data.

Technology Stack:

Programming language	Python 3.x
IDE	VS Code
Version Control	GitHub
Libraries in use	Matplotlib, NumPy, Pandas
API	Open-Meteo API (No API key needed)
GUI	PySide 6
Dataset Format	CSV

Design Diagrams:

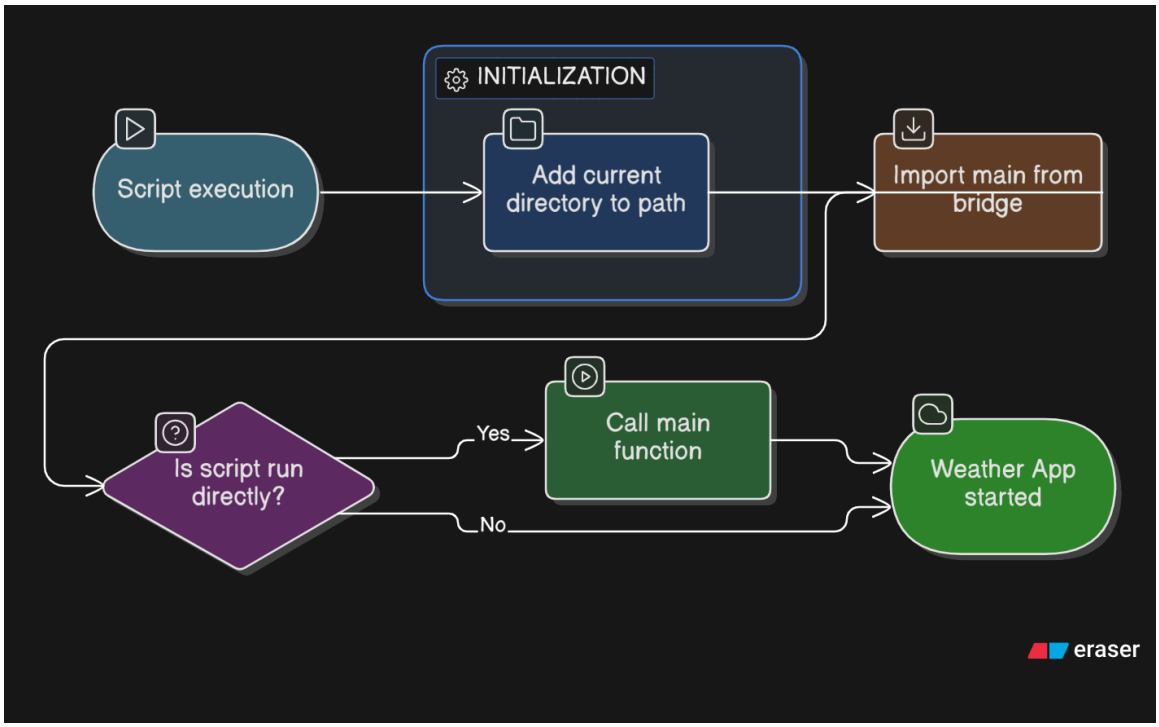


Fig. 1 This flowchart shows how the file runs and opens the program

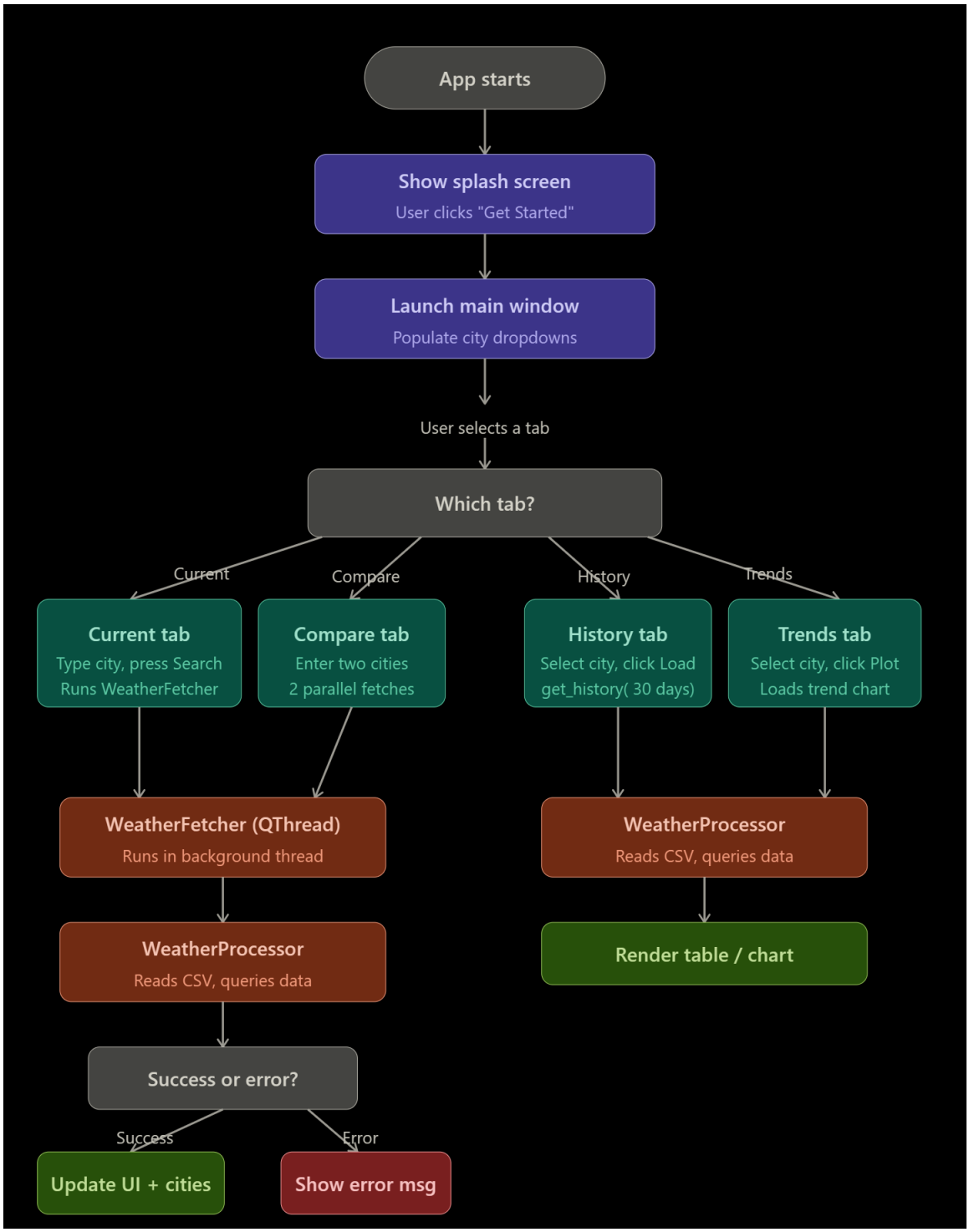


Fig. 2 The above diagram explains the functioning of the program in brief

Requirements to run the Program:

- Graphics & UI
  - PySide6 - 6.4.0

- Data Processing
  - Pandas - 1.5.0
  - NumPy - 1.23.0

- Networking & Time
  - Requests - 2.28.0
  - Python-dateutil - 2.8.2

- Visualization
  - Matplotlib - 3.6.0

The above resources are needed to be installed in order to run the program smoothly.

Source Code:

GUI Code

```
# Comments are added where necessary to understand the code in a better way
"""
weather_frontend.py
═══════════════════════════════════════════════════════════════════════════
FRONTEND LAYER — PySide6 Weather Application
─────────────────────────────────────────────────────────────────────────
This file is purely responsible for the UI. It owns:
  • All widget construction and layout
  • All visual styling (palette, stylesheets, custom-painted widgets)
  • The animated Aurora background
  • 4 tabs: Current, Compare, History, Trends

What this file does NOT do:
  • Make any network requests
  • Read/write any database
  • Hold any real weather data

═══════════════════════════════════════════════════════════════════════════
HOW TO CONNECT THE BACKEND
─────────────────────────────────────────────────────────────────────────
Every place the backend needs to push data is marked with:

# ──────────────────── BACKEND HOOK ────────────────────
```

Search that tag to find every integration point in one go.

Each hook documents:

- What data it expects (field names + types)
- Which method to call
- What the UI will do with the data

The general pattern everywhere is:

tab.load_data(<your_dict_here>)

═══

DEPENDENCIES

───

pip install PySide6 matplotlib numpy

═══

"""

```
import sys
from datetime import datetime
from PySide6.QtWidgets import (QApplication, QMainWindow, QWidget,
QVBoxLayout,
                               QHBoxLayout, QGridLayout, QLabel, QLineEdit,
                               QPushButton, QScrollArea, QFrame, QTabWidget,
                               QTableWidget, QTableWidgetItem, QHeaderView,
                               QComboBox, QFileDialog, QMessageBox, QSizePolicy)
from PySide6.QtCore    import Qt, QTimer
from PySide6.QtGui     import (QPainter, QColor, QFont, QLinearGradient,
```

```
                    QPen, QPainterPath, QRadialGradient)
import matplotlib
matplotlib.use("Qt5Agg")
from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg as FigureCanvas
from matplotlib.figure import Figure
import numpy as np

# ═══════════════════════════════════════════════════════════════
#  PALETTE
#  All colours live here. Change a value once and it updates everywhere.
# ═══════════════════════════════════════════════════════════════
BG_CARD   = QColor(10, 22, 44, 165)   # semi-transparent navy  — card backgrounds
BORDER    = QColor(90, 170, 255, 40)  # subtle blue            — default card borders
BORDER_HI = QColor(110, 195, 255, 85) # brighter blue          — glowing card border
ACC       = "#4fc3f7"               # sky blue               — accent / headings
TXT_PRI   = "#e8f4fd"               # near-white             — primary text
TXT_SEC   = "#7bafd4"               # muted blue             — secondary text
TXT_DIM   = "#2e5070"               # dark blue              — labels / dimmed text
DANGER    = "#ef5350"               # red                    — error / extreme heat
WARN      = "#ffb74d"               # amber                  — warnings / sunrise
FONT      = "Segoe UI"              # global font family

# ═══════════════════════════════════════════════════════════════
#  DESIGN PRIMITIVES
# ═══════════════════════════════════════════════════════════════
def L(text="", size=12, bold=False, color=TXT_PRI,
      align=Qt.AlignmentFlag.AlignLeft):
```

```
    l = QLabel(text)
    f = QFont(FONT, size)
    f.setBold(bold)
    l.setFont(f)
    l.setAlignment(align)
    l.setStyleSheet(f"color:{color}; background:transparent;")
    return l

class Frost(QWidget):
    def __init__(self, parent=None, r=18, glow=False):
        super().__init__(parent)
        self._r    = r
        self._glow = glow
        self.setAttribute(Qt.WidgetAttribute.WA_TranslucentBackground)
        # Allow the widget to grow/shrink with its layout contents
        self.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Preferred)

    def paintEvent(self, e):
        p = QPainter(self)
        p.setRenderHint(QPainter.RenderHint.Antialiasing)

        path = QPainterPath()
        path.addRoundedRect(0, 0, self.width(), self.height(), self._r, self._r)
        p.fillPath(path, BG_CARD)

        hi = QLinearGradient(0, 0, 0, 55)
        hi.setColorAt(0, QColor(255, 255, 255, 16))
```

```
        hi.setColorAt(1, QColor(255, 255, 255, 0))
        p.fillPath(path, hi)

        p.setPen(QPen(BORDER_HI if self._glow else BORDER, 1))
        p.drawPath(path)

        if self._glow:
            gp = QPainterPath()
            gp.addRoundedRect(-2, -2, self.width()+4, self.height()+4,
                              self._r+2, self._r+2)
            p.setPen(QPen(QColor(79, 195, 247, 28), 3))
            p.setBrush(Qt.BrushStyle.NoBrush)
            p.drawPath(gp)

SS = (
    f"QLineEdit{{background:rgba(6,16,34,210);"
    f"border:1px solid rgba(90,170,255,38);border-radius:12px;"
    f"padding:0 16px;color:{TXT_PRI};}}"
    f"QLineEdit:focus{{border:1.5px solid rgba(79,195,247,110);}}"
)

BSS = (
    f"QPushButton{{background:rgba(79,195,247,20);"
    f"border:1px solid rgba(79,195,247,55);border-radius:12px;"
    f"color:{ACC};font-family:{FONT};}}"
    f"QPushButton:hover{{background:rgba(79,195,247,38);}}"
    f"QPushButton:pressed{{background:rgba(79,195,247,12);}}"
```

```
)

CSS = (
    f"QComboBox{{background:rgba(6,16,34,210);"
    f"border:1px solid rgba(90,170,255,38);border-radius:10px;"
    f"padding:0 12px;color:{TXT_PRI};font-family:{FONT};}}"
    f"QComboBox::drop-down{{border:none;width:24px;}}"
    f"QComboBox QAbstractItemView{{background:#060f1e;color:{TXT_PRI};"
    f"border:1px solid rgba(79,195,247,45);"
    f"selection-background-color:rgba(79,195,247,28);}}"
)

class Pill(Frost):
    def __init__(self, emoji, label, value):
        super().__init__(r=14)
        self.setFixedHeight(72)
        lay = QVBoxLayout(self)
        lay.setContentsMargins(14, 8, 14, 8)
        lay.setSpacing(2)
        top = QHBoxLayout()
        el  = QLabel(emoji)
        el.setFont(QFont(FONT, 14))
        el.setStyleSheet("background:transparent;")
        top.addWidget(el)
        top.addWidget(L(label, 9, color=TXT_DIM))
        top.addStretch()
        self._v = L(value, 15, bold=True)
```

```
        lay.addLayout(top)
        lay.addWidget(self._v)

    def set(self, v):
        self._v.setText(v)

class HChip(Frost):
    def __init__(self, time, icon, temp):
        super().__init__(r=14)
        self.setFixedSize(82, 108)
        lay = QVBoxLayout(self)
        lay.setContentsMargins(4, 8, 4, 8)
        lay.setSpacing(3)
        lay.setAlignment(Qt.AlignmentFlag.AlignHCenter)
        lay.addWidget(L(time, 9, color=ACC, align=Qt.AlignmentFlag.AlignCenter))
        ic = QLabel(icon)
        ic.setFont(QFont(FONT, 20))
        ic.setAlignment(Qt.AlignmentFlag.AlignCenter)
        ic.setStyleSheet("background:transparent;")
        lay.addWidget(ic)
        lay.addWidget(L(f"{temp}°", 11, bold=True,
                        align=Qt.AlignmentFlag.AlignCenter))

class FRow(Frost):
    def __init__(self, day, icon, high, low, rain):
        super().__init__(r=12)
        self.setFixedHeight(52)
```

```
        lay = QHBoxLayout(self)
        lay.setContentsMargins(18, 0, 18, 0)
        dl = L(day, 12, bold=True)
        dl.setFixedWidth(40)
        ic = QLabel(icon)
        ic.setFont(QFont(FONT, 18))
        ic.setAlignment(Qt.AlignmentFlag.AlignCenter)
        ic.setStyleSheet("background:transparent;")
        ic.setFixedWidth(34)
        rl = L(f"💧{rain}%", 10, color=ACC)
        rl.setFixedWidth(56)
        lay.addWidget(dl)
        lay.addWidget(ic)
        lay.addWidget(rl)
        lay.addStretch()
        lay.addWidget(L(f"{low}°", 12, color=TXT_SEC,
                        align=Qt.AlignmentFlag.AlignRight))
        lay.addSpacing(6)
        lay.addWidget(L("·····", 9, color=TXT_DIM,
                        align=Qt.AlignmentFlag.AlignCenter))
        lay.addSpacing(6)
        lay.addWidget(L(f"{high}°", 13, bold=True,
                        align=Qt.AlignmentFlag.AlignRight))
# ═══════════════════════════════════════════════════════════════
#  AURORA BACKGROUND
# ═══════════════════════════════════════════════════════════════
```

```
class Aurora(QWidget):
    def __init__(self):
        super().__init__()
        self._t = 0.0
        t = QTimer(self)
        t.timeout.connect(self._tick)
        t.start(50)

    def _tick(self):
        self._t += 0.8
        self.update()

    def paintEvent(self, e):
        import math, random
        p = QPainter(self)
        p.setRenderHint(QPainter.RenderHint.Antialiasing)
        w, h = self.width(), self.height()

        base = QLinearGradient(0, 0, 0, h)
        base.setColorAt(0.0, QColor("#010a15"))
        base.setColorAt(0.5, QColor("#020e1c"))
        base.setColorAt(1.0, QColor("#010b17"))
        p.fillRect(self.rect(), base)

        bx = w*0.72 + math.sin(math.radians(self._t*0.55)) * 130
        by = h*0.20 + math.cos(math.radians(self._t*0.38)) * 65
        g  = QRadialGradient(bx, by, 370)
```

```
        g.setColorAt(0.0,  QColor(18,  85, 200, 42))
        g.setColorAt(0.55, QColor(10,  55, 150, 18))
        g.setColorAt(1.0,  QColor(0,    0,   0,  0))
        pp = QPainterPath()
        pp.addEllipse(bx-370, by-280, 740, 560)
        p.fillPath(pp, g)

        bx2 = w*0.20 + math.cos(math.radians(self._t*0.45)) * 110
        by2 = h*0.62 + math.sin(math.radians(self._t*0.32)) * 75
        g2  = QRadialGradient(bx2, by2, 290)
        g2.setColorAt(0.0, QColor(0, 130, 165, 30))
        g2.setColorAt(0.6, QColor(0,  80, 120, 12))
        g2.setColorAt(1.0, QColor(0,   0,   0,  0))
        pp2 = QPainterPath()
        pp2.addEllipse(bx2-290, by2-230, 580, 460)
        p.fillPath(pp2, g2)

        rng = random.Random(42)
        p.setPen(QPen(QColor(255, 255, 255, 28), 1))
        for _ in range(55):
            p.drawPoint(rng.randint(0, w), rng.randint(0, h))

# ═══════════════════════════════════════════════════════════════
#  TAB 1 — CURRENT WEATHER
# ═══════════════════════════════════════════════════════════════
class CurrentTab(QWidget):
    def __init__(self):
```

```
        super().__init__()
        self.setAttribute(Qt.WidgetAttribute.WA_TranslucentBackground)
        self._build()
        self._pending_data = None

    def _build(self):
        outer = QVBoxLayout(self)
        outer.setContentsMargins(0, 0, 0, 0)

        sc = QScrollArea()
        sc.setWidgetResizable(True)
        sc.setFrameShape(QFrame.Shape.NoFrame)
        sc.setStyleSheet("background:transparent; border:none;")
        sc.setHorizontalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAlwaysOff)
        outer.addWidget(sc)

        body = QWidget()
        body.setStyleSheet("background:transparent;")
        sc.setWidget(body)
        lay = QVBoxLayout(body)
        lay.setSpacing(14)
        lay.setContentsMargins(20, 20, 20, 28)

        # ── Search bar ──────────────────────────────────────────
        row = QHBoxLayout(); row.setSpacing(8)
        self.search = QLineEdit()
        self.search.setPlaceholderText("Search any city in the world…")
```

```
self.search.setFixedHeight(46)
self.search.setFont(QFont(FONT, 12))
self.search.setStyleSheet(SS)
self.search.returnPressed.connect(self._fetch)
btn = QPushButton("Search")
btn.setFixedSize(100, 46)
btn.setFont(QFont(FONT, 12))
btn.setStyleSheet(BSS)
btn.clicked.connect(self._fetch)
row.addWidget(self.search)
row.addWidget(btn)
lay.addLayout(row)

self.status = L("", 10, color=DANGER, align=Qt.AlignmentFlag.AlignCenter)
lay.addWidget(self.status)

# ── Hero card ──────────────────────────────────────────────
hero = Frost(r=22, glow=True)
hero.setMinimumHeight(228)
hl = QVBoxLayout(hero)
hl.setContentsMargins(24, 18, 24, 18)
hl.setSpacing(8)

cr = QHBoxLayout()
self.city_lbl = L("—", 20, bold=True)
self.date_lbl = L("", 10, color=TXT_DIM)
cr.addWidget(self.city_lbl)
```

```
cr.addStretch()
cr.addWidget(self.date_lbl)
hl.addLayout(cr)

self.cond_lbl = L("Enter a city to get live weather", 11, color=TXT_SEC)
hl.addWidget(self.cond_lbl)
hl.addSpacing(4)

mid = QHBoxLayout()
self.icon_lbl = QLabel("🌐")
self.icon_lbl.setFont(QFont(FONT, 56))
self.icon_lbl.setStyleSheet("background:transparent;")
self.temp_lbl = L("—", 56, bold=True)
mid.addWidget(self.icon_lbl)
mid.addWidget(self.temp_lbl)
mid.addStretch()
rhs = QVBoxLayout()
rhs.setAlignment(Qt.AlignmentFlag.AlignVCenter)
rhs.setSpacing(6)
self.feels_lbl = L("Feels like —", 11, color=TXT_SEC)
self.uv_lbl    = L("UV Index  —", 11, color=TXT_SEC)
rhs.addWidget(self.feels_lbl)
rhs.addWidget(self.uv_lbl)
mid.addLayout(rhs)
hl.addLayout(mid)
hl.addStretch()
```

```
sun = QHBoxLayout()
self.sr_lbl = L("🌅 —", 11, color=WARN)
self.ss_lbl = L("— 🌇", 11, color="#ff8a65",
                align=Qt.AlignmentFlag.AlignRight)
sun.addWidget(self.sr_lbl)
sun.addStretch()
sun.addWidget(self.ss_lbl)
hl.addLayout(sun)
lay.addWidget(hero)

# ── Stat pills 2x2 grid ──────────────────────────────────────
grid = QGridLayout()
grid.setSpacing(10)
self.pills = {
    "humidity":   Pill("💧", "Humidity",   "—"),
    "wind":       Pill("💨", "Wind",       "—"),
    "pressure":   Pill("🌡", "Pressure",   "—"),
    "visibility": Pill("👁", "Visibility", "—"),
}
for i, w in enumerate(self.pills.values()):
    grid.addWidget(w, i // 2, i % 2)
lay.addLayout(grid)

# ── Alert banner ─────────────────────────────────────────────
self.alert = Frost(r=12)
```

```
        al = QHBoxLayout(self.alert)
        al.setContentsMargins(16, 10, 16, 10)
        al.setSpacing(10)
        warn_icon = L("⚠", 14)
        warn_icon.setFixedWidth(24)
        al.addWidget(warn_icon)
        self.alert_lbl = L("", 11, color="#ff8a80")
        self.alert_lbl.setWordWrap(True)
        al.addWidget(self.alert_lbl, 1)
        self.alert.hide()
        lay.addWidget(self.alert)

        # ── Hourly forecast horizontal scroll ──────────────────────────
        # FIX 1: use a proper container widget that grows with content
        lay.addWidget(L("  Hourly Forecast", 12, bold=True, color=ACC))
        hs = QScrollArea()
        hs.setFixedHeight(128)
        hs.setFrameShape(QFrame.Shape.NoFrame)
        hs.setVerticalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAlwaysOff)
        hs.setHorizontalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAsNeeded)
        hs.setStyleSheet("background:transparent; border:none;")
        hs.horizontalScrollBar().setStyleSheet(
            "QScrollBar:horizontal{background:transparent;height:4px;}"
            "QScrollBar::handle:horizontal{background:rgba(79,195,247,40);border-
radius:2px;}"
            "QScrollBar::add-line:horizontal,QScrollBar::sub-line:horizontal{width:0px;}"
        )
```

```
        self._hw = QWidget()
        self._hw.setStyleSheet("background:transparent;")
        self._hw.setFixedHeight(116)  # set height NOW before setWidget
        self._hrow = QHBoxLayout(self._hw)
        self._hrow.setSpacing(8)
        self._hrow.setContentsMargins(4, 4, 4, 4)
        self._hrow.setAlignment(Qt.AlignmentFlag.AlignLeft |
Qt.AlignmentFlag.AlignVCenter)
        hs.setWidget(self._hw)
        hs.setWidgetResizable(False)  # False = we control width manually in load_data
        lay.addWidget(hs)
        self._hs = hs

        # ── 7-day forecast card ──────────────────────────────────────
        lay.addWidget(L("  7-Day Forecast", 12, bold=True, color=ACC))
        self._fc = Frost(r=18)
        self._fc.setSizePolicy(QSizePolicy.Policy.Expanding,
QSizePolicy.Policy.Minimum)
        self._fl = QVBoxLayout(self._fc)
        self._fl.setContentsMargins(8, 10, 8, 10)
        self._fl.setSpacing(4)
        lay.addWidget(self._fc)
        lay.addStretch()

    def _fetch(self):
        city = self.search.text().strip()
        if not city:
```

```
        return
    self.show_loading()

def show_loading(self):
    self.status.setStyleSheet(f"color:{ACC};")
    self.status.setText("Fetching weather data…")

def show_error(self, message: str):
    self.status.setStyleSheet(f"color:{DANGER};")
    self.status.setText(message)

def load_data(self, data: dict):
    print("□ load_data START", flush=True)

    try:
        self.status.setText("")

        self.city_lbl.setText(f"📍 {data['city']}, {data['country']}")
        self.date_lbl.setText(datetime.now().strftime("%a %d %b %Y"))
        self.cond_lbl.setText(data["condition"])
        self.icon_lbl.setText(data["icon"])
        self.temp_lbl.setText(f"{data['temp']}°C")
        self.feels_lbl.setText(f"Feels like  {data['feels_like']}°C")
        self.uv_lbl.setText(f"UV Index  {data['uv_index']}")
        self.sr_lbl.setText(f"🌅  {data['sunrise']}")
        self.ss_lbl.setText(f"{data['sunset']}  🌇")
```

```
        self.pills["humidity"].set(f"{data['humidity']}%")
        self.pills["wind"].set(f"{data['wind']} km/h")
        self.pills["pressure"].set(f"{data['pressure']} hPa")
        self.pills["visibility"].set(f"{data['visibility']} km")

        # ── hourly chips ──────────────────────────────────────────
        # deleteLater() is async — hide first so old chips vanish immediately
        for i in range(self._hrow.count()):
            item = self._hrow.itemAt(i)
            if item and item.widget():
                item.widget().hide()
        while self._hrow.count():
            item = self._hrow.takeAt(0)
            if item.widget():
                item.widget().deleteLater()

        hourly = data.get("hourly", [])
        CHIP_W  = 82   # matches HChip.setFixedSize
        CHIP_SP = 8    # matches _hrow spacing
        MARGIN  = 16

        for h in hourly:
            chip = HChip(h["time"], h["icon"], h["temp"])
            self._hrow.addWidget(chip)
            chip.show()   # force visible — deleteLater from prev pass can hide new
widgets
```

```
# widgetResizable=False means we MUST set width manually
total_w = len(hourly) * (CHIP_W + CHIP_SP) + MARGIN
self._hw.setMinimumWidth(total_w)
self._hw.setFixedHeight(116)
self._hw.updateGeometry()

print(f"□ Added {len(hourly)} hourly chips, container={total_w}px", flush=True)

# ── 7-day forecast rows ──────────────────────────────────────
for i in range(self._fl.count()):
    item = self._fl.itemAt(i)
    if item and item.widget():
        item.widget().hide()
while self._fl.count():
    item = self._fl.takeAt(0)
    if item.widget():
        item.widget().deleteLater()

forecast = data.get("forecast", [])
row_h = 52   # matches FRow.setFixedHeight
sep_h = 2
needed_h = len(forecast) * row_h + (len(forecast) - 1) * sep_h + 20
self._fc.setMinimumHeight(max(needed_h, 60))

for i, f in enumerate(forecast):
```

```
    frow = FRow(f["day"], f["icon"], f["high"], f["low"], f["rain"])
    self._fl.addWidget(frow)
    frow.show()
    if i < len(forecast) - 1:
        ln = QFrame()
        ln.setFrameShape(QFrame.Shape.HLine)
        ln.setStyleSheet("color:rgba(79,195,247,14);")
        self._fl.addWidget(ln)

self._fc.updateGeometry()

print(f"□ Added {len(forecast)} forecast rows", flush=True)

alerts = []
if data["temp"] >= 40: alerts.append("🔥 Extreme heat")
if data["wind"] >= 50: alerts.append("💨 High winds")
if data["humidity"] >= 90: alerts.append("💧 Very high humidity")
if data["uv_index"] >= 8: alerts.append("☀ High UV")
if alerts:
    self.alert_lbl.setText("  ·  ".join(alerts))
    self.alert.show()
else:
    self.alert.hide()

print("□ load_data COMPLETE!", flush=True)
```

```
        except Exception as e:
            print(f"❌ Error in load_data: {e}", flush=True)
            import traceback
            traceback.print_exc()
            self.show_error(f"Error: {e}")

# ═══════════════════════════════════════════════════════════════
#  TAB 2 — COMPARE
# ═══════════════════════════════════════════════════════════════
class CompareTab(QWidget):
    def __init__(self):
        super().__init__()
        self.setAttribute(Qt.WidgetAttribute.WA_TranslucentBackground)
        self._d1 = None
        self._d2 = None
        self._build()

    def _build(self):
        lay = QVBoxLayout(self)
        lay.setContentsMargins(20, 20, 20, 20)
        lay.setSpacing(14)
        lay.addWidget(L("Compare Two Cities", 16, bold=True, color=ACC,
                        align=Qt.AlignmentFlag.AlignCenter))

        row = QHBoxLayout(); row.setSpacing(8)
        self.c1 = QLineEdit(); self.c1.setPlaceholderText("City 1")
```

```
self.c2 = QLineEdit(); self.c2.setPlaceholderText("City 2")
for e in (self.c1, self.c2):
    e.setFixedHeight(42); e.setFont(QFont(FONT, 12)); e.setStyleSheet(SS)
btn = QPushButton("Compare")
btn.setFixedHeight(42); btn.setFont(QFont(FONT, 12)); btn.setStyleSheet(BSS)
btn.clicked.connect(self._compare)
row.addWidget(self.c1); row.addWidget(self.c2); row.addWidget(btn)
lay.addLayout(row)

self.status = L("", 10, color=DANGER, align=Qt.AlignmentFlag.AlignCenter)
lay.addWidget(self.status)

cards = QHBoxLayout(); cards.setSpacing(12)
self.cd1 = self._mk_card()
self.cd2 = self._mk_card()
vs = L("VS", 20, bold=True, color=TXT_DIM,
       align=Qt.AlignmentFlag.AlignCenter)
vs.setFixedWidth(38)
cards.addWidget(self.cd1); cards.addWidget(vs); cards.addWidget(self.cd2)
lay.addLayout(cards)

self.fig    = Figure(figsize=(6, 2.8), facecolor="none")
self.canvas = FigureCanvas(self.fig)
self.canvas.setStyleSheet("background:transparent;")
self.canvas.setMinimumHeight(200)
lay.addWidget(self.canvas)
```

```
        self.banner = Frost(r=12)
        bl = QHBoxLayout(self.banner)
        bl.setContentsMargins(16, 10, 16, 10)
        self.blbl = L("", 11, color=TXT_SEC, align=Qt.AlignmentFlag.AlignCenter)
        self.blbl.setWordWrap(True)
        bl.addWidget(self.blbl)
        self.banner.hide()
        lay.addWidget(self.banner)
        lay.addStretch()

    def _mk_card(self):
        c  = Frost(r=16)
        cl = QVBoxLayout(c)
        cl.setContentsMargins(14, 14, 14, 14)
        cl.setSpacing(6)
        c._n = L("—", 13, bold=True, align=Qt.AlignmentFlag.AlignCenter)
        c._i = QLabel("🌐")
        c._i.setFont(QFont(FONT, 38))
        c._i.setAlignment(Qt.AlignmentFlag.AlignCenter)
        c._i.setStyleSheet("background:transparent;")
        c._t = L("—°C", 26, bold=True, align=Qt.AlignmentFlag.AlignCenter)
        c._c = L("—", 10, color=TXT_SEC, align=Qt.AlignmentFlag.AlignCenter)
        c._s = L("", 10, color=TXT_DIM,  align=Qt.AlignmentFlag.AlignCenter)
        c._s.setWordWrap(True)
        for w in (c._n, c._i, c._t, c._c, c._s):
            cl.addWidget(w)
```

```
        return c

    def _fill_card(self, card, city: str, data: dict):
        card._n.setText(city)
        card._i.setText(data["icon"])
        card._t.setText(f"{data['temp']}°C")
        card._c.setText(data["condition"])
        card._s.setText(
            f"💧{data['humidity']}%  🌬{data['wind']} km/h\n"
            f"🌡 {data['pressure']} hPa  👁{data['visibility']} km"
        )

    def _compare(self):
        c1 = self.c1.text().strip()
        c2 = self.c2.text().strip()
        if not c1 or not c2:
            self.status.setText("Enter both cities.")
            return
        self.show_loading()
        self._d1 = None
        self._d2 = None

    def show_loading(self):
        self.status.setStyleSheet(f"color:{ACC};")
        self.status.setText("Fetching…")
```

```
def show_error(self, message: str):
    self.status.setStyleSheet(f"color:{DANGER};")
    self.status.setText(message)

def load_city1(self, city: str, data: dict):
    self._d1 = (city, data)
    self._fill_card(self.cd1, city, data)
    self._try_draw()

def load_city2(self, city: str, data: dict):
    self._d2 = (city, data)
    self._fill_card(self.cd2, city, data)
    self._try_draw()

def _try_draw(self):
    if not (self._d1 and self._d2):
        return
    self.status.setText("")
    c1, d1 = self._d1
    c2, d2 = self._d2

    metrics = ["Temp °C", "Humidity", "Wind", "UV"]
    v1 = [d1["temp"], d1["humidity"], d1["wind"], d1["uv_index"]]
    v2 = [d2["temp"], d2["humidity"], d2["wind"], d2["uv_index"]]
    self.fig.clear()
    ax = self.fig.add_subplot(111)
    self.fig.patch.set_alpha(0)
```

```
ax.set_facecolor("#ffffff04")
x  = np.arange(len(metrics)); bw = 0.32
b1 = ax.bar(x - bw/2, v1, bw, label=c1, color="#4fc3f7", alpha=0.82, zorder=3)
b2 = ax.bar(x + bw/2, v2, bw, label=c2, color="#11515a", alpha=0.82, zorder=3)
ax.set_xticks(x)
ax.set_xticklabels(metrics, color=TXT_SEC, fontsize=9)
ax.tick_params(axis="y", colors=TXT_SEC, labelsize=8)
ax.spines[:].set_visible(False)
ax.yaxis.grid(True, color="#ffffff0c", zorder=0)
ax.legend(facecolor="#060f1e", edgecolor="#1a3a5a",
          labelcolor=TXT_PRI, fontsize=9)
for b in (*b1, *b2):
    h = b.get_height()
    ax.text(b.get_x() + b.get_width()/2, h + .3, f"{h:.0f}",
            ha="center", va="bottom", color=TXT_SEC, fontsize=8)
self.canvas.draw()


ht = c1 if d1["temp"]     > d2["temp"]     else c2
wd = c1 if d1["wind"]     > d2["wind"]     else c2
hm = c1 if d1["humidity"] > d2["humidity"] else c2
self.blbl.setText(
    f" 🌡 Hotter: {ht} ({max(d1['temp'], d2['temp'])}°C)   "
    f"·  🌬 Windier: {wd}  ·  💧 More humid: {hm}"
)
self.banner.show()
```

```
# ═══════════════════════════════════════════════════════════════
#  TAB 3 — HISTORY
# ═══════════════════════════════════════════════════════════════
class HistoryTab(QWidget):
    def __init__(self):
        super().__init__()
        self.setAttribute(Qt.WidgetAttribute.WA_TranslucentBackground)
        self._build()

    def _build(self):
        lay = QVBoxLayout(self)
        lay.setContentsMargins(20, 20, 20, 20)
        lay.setSpacing(12)
        lay.addWidget(L("Weather History", 16, bold=True, color=ACC,
                        align=Qt.AlignmentFlag.AlignCenter))

        row = QHBoxLayout(); row.setSpacing(8)
        self.combo = QComboBox()
        self.combo.setFixedHeight(38)
        self.combo.setFont(QFont(FONT, 11))
        self.combo.setStyleSheet(CSS)
        btn = QPushButton("Load")
        btn.setFixedHeight(38); btn.setFont(QFont(FONT, 11)); btn.setStyleSheet(BSS)
        btn.clicked.connect(lambda: self._load())
        row.addWidget(L("City:", 11)); row.addWidget(self.combo); row.addWidget(btn)
        lay.addLayout(row)
```

```
hc = QHBoxLayout(); hc.setSpacing(12)
self.hot  = self._mk_extreme_card("🔥", "Hottest Day", DANGER)
self.cold = self._mk_extreme_card("❄", "Coldest Day", ACC)
hc.addWidget(self.hot); hc.addWidget(self.cold)
lay.addLayout(hc)

self.table = QTableWidget()
self.table.setColumnCount(6)
self.table.setHorizontalHeaderLabels(
   ["Date", "Temp °C", "Humidity", "Pressure", "Wind", "Condition"])
self.table.horizontalHeader().setSectionResizeMode(
   QHeaderView.ResizeMode.Stretch)
self.table.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)
self.table.setAlternatingRowColors(True)
self.table.verticalHeader().setVisible(False)
self.table.setStyleSheet(f"""
   QTableWidget {{
      background:rgba(5,13,26,175);
      border:1px solid rgba(79,195,247,28);
      border-radius:12px; color:{TXT_PRI};
      gridline-color:rgba(79,195,247,10);
   }}
   QHeaderView::section {{
      background:rgba(79,195,247,16); color:{ACC};
      border:none; padding:6px;
      font-size:10px; font-weight:bold;
```

```
        }}
        QTableWidget::item:alternate {{background:rgba(79,195,247,5);}}
        QScrollBar:vertical {{background:transparent; width:6px;}}
        QScrollBar::handle:vertical {{
            background:rgba(79,195,247,38); border-radius:3px;}}
    """)
    lay.addWidget(self.table)

    exp = QPushButton("📥  Export to CSV")
    exp.setFixedHeight(36); exp.setFont(QFont(FONT, 11)); exp.setStyleSheet(BSS)
    exp.clicked.connect(self._export)
    lay.addWidget(exp, alignment=Qt.AlignmentFlag.AlignRight)

def _mk_extreme_card(self, emoji, title, color):
    c  = Frost(r=14)
    cl = QVBoxLayout(c)
    cl.setContentsMargins(16, 12, 16, 12)
    cl.setSpacing(4)
    hr = QHBoxLayout()
    el = QLabel(emoji)
    el.setFont(QFont(FONT, 16))
    el.setStyleSheet("background:transparent;")
    hr.addWidget(el)
    hr.addWidget(L(title, 11, color=color))
    hr.addStretch()
    c._t = L("—", 22, bold=True, color=color)
```

```
    c._d = L("—", 9,  color=TXT_DIM)
    c._c = L("—", 10, color=TXT_SEC)
    cl.addLayout(hr)
    cl.addWidget(c._t)
    cl.addWidget(c._d)
    cl.addWidget(c._c)
    return c

def set_cities(self, cities: list):
    self.combo.clear()
    for city in cities:
        self.combo.addItem(city)

def _load(self):
    city = self.combo.currentText()
    if not city:
        return

def load_history(self, rows: list):
    self.table.setRowCount(len(rows))
    for r, row in enumerate(rows):
        for c, val in enumerate(row):
            item = QTableWidgetItem(str(val))
            item.setTextAlignment(Qt.AlignmentFlag.AlignCenter)
            self.table.setItem(r, c, item)

    if rows:
```

```
        hot  = max(rows, key=lambda x: x[1])
        cold = min(rows, key=lambda x: x[1])
        self.hot._t.setText(f"{hot[1]}°C")
        self.hot._d.setText(hot[0])
        self.hot._c.setText(hot[5])
        self.cold._t.setText(f"{cold[1]}°C")
        self.cold._d.setText(cold[0])
        self.cold._c.setText(cold[5])

def _export(self):
    city = self.combo.currentText()
    if not city:
        return
    path, _ = QFileDialog.getSaveFileName(
        self, "Save CSV", f"{city}_weather.csv", "CSV files (*.csv)")
    if not path:
        return
    import csv
    with open(path, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["Date", "Temp", "Humidity", "Pressure", "Wind", "Condition"])
        for r in range(self.table.rowCount()):
            writer.writerow([
                self.table.item(r, c).text()
                for c in range(self.table.columnCount())
            ])
    QMessageBox.information(self, "Done", f"Saved:\n{path}")
```

```
# ═══════════════════════════════════════════════════════════
#  TAB 4 — TRENDS
# ═══════════════════════════════════════════════════════════
class TrendsTab(QWidget):
    def __init__(self):
        super().__init__()
        self.setAttribute(Qt.WidgetAttribute.WA_TranslucentBackground)
        self._rows = []
        self._build()

    def _build(self):
        lay = QVBoxLayout(self)
        lay.setContentsMargins(20, 20, 20, 20)
        lay.setSpacing(12)
        lay.addWidget(L("Trend Analysis", 16, bold=True, color=ACC,
                        align=Qt.AlignmentFlag.AlignCenter))

        ctrl = QHBoxLayout(); ctrl.setSpacing(8)
        self.cc = QComboBox()
        self.cc.setFixedHeight(36); self.cc.setStyleSheet(CSS);
self.cc.setFont(QFont(FONT, 11))
        self.mc = QComboBox()
        self.mc.addItems(["Temperature", "Humidity", "Pressure", "Wind Speed"])
        self.mc.setFixedHeight(36); self.mc.setStyleSheet(CSS);
self.mc.setFont(QFont(FONT, 11))
        btn = QPushButton("Plot")
```

```
    btn.setFixedHeight(36); btn.setStyleSheet(BSS); btn.setFont(QFont(FONT, 11))
    btn.clicked.connect(lambda: self._plot())
    ctrl.addWidget(L("City:", 11)); ctrl.addWidget(self.cc)
    ctrl.addWidget(L("Metric:", 11)); ctrl.addWidget(self.mc)
    ctrl.addWidget(btn)
    lay.addLayout(ctrl)

    self.fig    = Figure(figsize=(7, 3.2), facecolor="none")
    self.canvas = FigureCanvas(self.fig)
    self.canvas.setStyleSheet("background:transparent;")
    lay.addWidget(self.canvas)

    pr = QHBoxLayout(); pr.setSpacing(10)
    self.pa  = Pill("📊", "30-Day Avg", "—")
    self.pmx = Pill("↑",  "Max",       "—")
    self.pmn = Pill("↓",  "Min",       "—")
    self.pt  = Pill("📈", "Trend",     "—")
    for p in (self.pa, self.pmx, self.pmn, self.pt):
        pr.addWidget(p)
    lay.addLayout(pr)
    lay.addStretch()

def set_cities(self, cities: list):
    self.cc.clear()
    for city in cities:
        self.cc.addItem(city)
```

```
def _plot(self):
    city = self.cc.currentText()
    if not city:
        return
    if self._rows:
        self._render()

def load_trends(self, rows: list):
    self._rows = rows
    self._render()

def _render(self):
    rows = self._rows
    if not rows:
        return

    metric = self.mc.currentText()
    cm = {
        "Temperature": (1, "°C",    "#4fc3f7"),
        "Humidity":    (2, "%",     "#26c6da"),
        "Pressure":    (3, " hPa",  "#ffb74d"),
        "Wind Speed":  (4, " km/h", "#66bb6a"),
    }
    col, unit, color = cm[metric]
    dates  = [r[0][-5:] for r in rows]
    values = [r[col]    for r in rows]
```

```
self.fig.clear()
ax = self.fig.add_subplot(111)
self.fig.patch.set_alpha(0)
ax.set_facecolor("#ffffff04")

ax.fill_between(range(len(values)), values, alpha=0.14, color=color)
ax.plot(range(len(values)), values, color=color,
        linewidth=2, marker="o", markersize=3.5, zorder=4)

if len(values) > 2:
    z = np.polyfit(range(len(values)), values, 1)
    poly = np.poly1d(z)
    ax.plot(range(len(values)), poly(range(len(values))),
            "--", color="#ffffff28", linewidth=1.2, zorder=3)
    slope = z[0]
    trend = f"↑ +{slope:.2f}{unit}/day" if slope > 0 else f"↓ {slope:.2f}{unit}/day"
else:
    trend = "N/A"

step  = max(1, len(dates) // 6)
ticks = list(range(0, len(dates), step))
ax.set_xticks(ticks)
ax.set_xticklabels([dates[i] for i in ticks],
                   color=TXT_SEC, fontsize=8, rotation=30)
ax.tick_params(axis="y", colors=TXT_SEC, labelsize=8)
ax.spines[:].set_visible(False)
```

```
        ax.yaxis.grid(True, color="#ffffff0c", zorder=0)
        ax.set_title(
            f"{self.cc.currentText()} — {metric} (last {len(values)} days)",
            color=ACC, fontsize=10, pad=8)
        self.canvas.draw()

        avg_ = sum(values) / len(values)
        self.pa.set(f"{avg_:.1f}{unit}")
        self.pmx.set(f"{max(values):.1f}{unit}")
        self.pmn.set(f"{min(values):.1f}{unit}")
        self.pt.set(trend)

# ═══════════════════════════════════════════════════════════════
#  MAIN WINDOW
# ═══════════════════════════════════════════════════════════════
class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Weather Forecast")
        self.setMinimumSize(800, 700)

        bg = Aurora()
        self.setCentralWidget(bg)
        outer = QVBoxLayout(bg)
        outer.setContentsMargins(0, 0, 0, 0)

        tabs = QTabWidget()
```

```
tabs.setStyleSheet(f"""
    QTabWidget::pane {{ background:transparent; border:none; }}
    QTabBar {{ background:transparent; }}
    QTabBar::tab {{
        background:rgba(5,13,26,165); color:{TXT_DIM};
        border:1px solid rgba(79,195,247,18); border-radius:10px;
        padding:9px 20px; margin:6px 3px 0 3px; font:12px '{FONT}';
    }}
    QTabBar::tab:selected {{
        background:rgba(79,195,247,18); color:{ACC};
        border:1px solid rgba(79,195,247,65);
    }}
    QTabBar::tab:hover {{ background:rgba(79,195,247,10); color:{TXT_SEC}; }}
    QScrollBar:vertical {{ background:transparent; width:6px; }}
    QScrollBar::handle:vertical {{
        background:rgba(79,195,247,35); border-radius:3px; }}
""")
outer.addWidget(tabs)

self.tc  = CurrentTab()
self.tcm = CompareTab()
self.th  = HistoryTab()
self.tt  = TrendsTab()

tabs.addTab(self.tc,  "⛅ Current")
tabs.addTab(self.tcm, "⤮ Compare")
```

```
        tabs.addTab(self.th,  "📋 History")
        tabs.addTab(self.tt,  "📈 Trends")

# ═══════════════════════════════════════════════════════════
#  SPLASH SCREEN
# ═══════════════════════════════════════════════════════════
class SplashBg(QWidget):
    def __init__(self):
        super().__init__()
        self._t = 0.0
        t = QTimer(self)
        t.timeout.connect(self._tick)
        t.start(50)

    def _tick(self):
        self._t += 0.5
        self.update()

    def paintEvent(self, e):
        import math
        p = QPainter(self)
        p.setRenderHint(QPainter.RenderHint.Antialiasing)
        w, h = self.width(), self.height()

        base = QLinearGradient(0, 0, 0, h)
        base.setColorAt(0.00, QColor(72,  40,  90))
```

```
base.setColorAt(0.35, QColor(160,  70,  90))
base.setColorAt(0.65, QColor(210, 110,  70))
base.setColorAt(1.00, QColor(180,  80,  60))
p.fillRect(self.rect(), base)

bx = w * 0.55 + math.sin(math.radians(self._t * 0.4)) * 80
by = h * 0.25 + math.cos(math.radians(self._t * 0.3)) * 40
g  = QRadialGradient(bx, by, 420)
g.setColorAt(0.00, QColor(240, 160,  60, 55))
g.setColorAt(0.50, QColor(200, 100,  50, 22))
g.setColorAt(1.00, QColor(0,     0,   0,  0))
pp = QPainterPath()
pp.addEllipse(bx - 420, by - 300, 840, 600)
p.fillPath(pp, g)

bx2 = w * 0.25 + math.cos(math.radians(self._t * 0.35)) * 70
by2 = h * 0.70 + math.sin(math.radians(self._t * 0.28)) * 50
g2  = QRadialGradient(bx2, by2, 320)
g2.setColorAt(0.00, QColor(180,  80, 120, 45))
g2.setColorAt(0.55, QColor(130,  50,  90, 18))
g2.setColorAt(1.00, QColor(0,     0,   0,  0))
pp2 = QPainterPath()
pp2.addEllipse(bx2 - 320, by2 - 240, 640, 480)
p.fillPath(pp2, g2)

vig = QRadialGradient(w / 2, h / 2, max(w, h) * 0.75)
vig.setColorAt(0.0, QColor(0, 0, 0,   0))
```

```
        vig.setColorAt(1.0, QColor(0, 0, 0,  90))
        pp3 = QPainterPath()
        pp3.addRect(0, 0, w, h)
        p.fillPath(pp3, vig)

class SplashScreen(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Mausam")
        self.setMinimumSize(860, 500)
        self.setFixedSize(860, 500)

        bg = SplashBg()
        self.setCentralWidget(bg)

        outer = QVBoxLayout(bg)
        outer.setContentsMargins(0, 0, 0, 0)
        outer.addStretch(2)

        name = QLabel("Mausam")
        name_font = QFont("Georgia", 58)
        name_font.setBold(True)
        name_font.setItalic(True)
        name.setFont(name_font)
        name.setAlignment(Qt.AlignmentFlag.AlignCenter)
        name.setStyleSheet("""
            color: white;
```

```
    background: transparent;
    letter-spacing: 3px;
""")
outer.addWidget(name)

outer.addSpacing(18)

tagline = QLabel(
    "Welcome to Mausam — a platform to get the real-time\n"
    "weather updates of any location of your choice, anytime!"
)
tag_font = QFont("Georgia", 12)
tagline.setFont(tag_font)
tagline.setAlignment(Qt.AlignmentFlag.AlignCenter)
tagline.setStyleSheet("""
    color: rgba(255, 240, 230, 210);
    background: transparent;
    line-height: 1.6;
""")
outer.addWidget(tagline)

outer.addSpacing(42)

btn_row = QHBoxLayout()
btn_row.addStretch()
self.btn = QPushButton("Get Started")
self.btn.setFixedSize(160, 44)
```

```
self.btn.setFont(QFont("Georgia", 12))
self.btn.setCursor(Qt.CursorShape.PointingHandCursor)
self.btn.setStyleSheet("""
    QPushButton {
        background: rgba(40, 30, 60, 200);
        color: white;
        border: 1.5px solid rgba(255, 255, 255, 80);
        border-radius: 6px;
        letter-spacing: 1px;
    }
    QPushButton:hover {
        background: rgba(60, 45, 85, 220);
        border: 1.5px solid rgba(255, 255, 255, 140);
    }
    QPushButton:pressed {
        background: rgba(25, 18, 45, 220);
    }
""")
self.btn.clicked.connect(self._launch)
btn_row.addWidget(self.btn)
btn_row.addStretch()
outer.addLayout(btn_row)

outer.addStretch(3)

footer = QLabel("Real-time weather · Trends · History · Compare")
footer.setFont(QFont("Segoe UI", 8))
```

```
        footer.setAlignment(Qt.AlignmentFlag.AlignCenter)
        footer.setStyleSheet("color: rgba(255,220,200,100); background:transparent;")
        outer.addWidget(footer)
        outer.addSpacing(14)

    def _launch(self):
        self._main = MainWindow()
        self._main.show()
        self.close()

# ═══════════════════════════════════════════════════════════════
#  ENTRY POINT
# ═══════════════════════════════════════════════════════════════
if __name__ == "__main__":
    app = QApplication(sys.argv)
    app.setStyle("Fusion")
    splash = SplashScreen()
    splash.show()
    sys.exit(app.exec())
```

Backend Code

```
# finalBackend.py - FIXED VERSION
"""
Weather Monitoring Backend Module
Uses Open-Meteo API (FREE, no API key needed!)
"""

import requests
from datetime import datetime, timedelta
from typing import Dict, List
import logging
import pandas as pd
import os

logging.basicConfig(level=logging.INFO, format='%(levelname)s: %(message)s')

COLUMNS = ["timestamp", "location_id", "temp_c", "humidity_pct",
           "wind_kph", "condition", "pressure_hpa"]

def _load_df(file_path: str) -> pd.DataFrame:
    """Load CSV and always return a DataFrame with a proper datetime timestamp column."""
    if os.path.exists(file_path):
        df = pd.read_csv(file_path, parse_dates=['timestamp'])
        # If parse_dates silently failed (column missing / bad data), coerce manually
        if 'timestamp' in df.columns and not pd.api.types.is_datetime64_any_dtype(df['timestamp']):
```

```
            df['timestamp'] = pd.to_datetime(df['timestamp'], errors='coerce')
        # Drop rows with missing critical fields
        df = df.dropna(subset=['location_id', 'temp_c'])
        return df
    return pd.DataFrame(columns=COLUMNS)

# ================ TELEMETRY STORAGE ENGINE ================
class TelemetryStorageEngine:
    def __init__(self, file_path: str = 'telemetry_logs/daily_telemetry.csv'):
        self.file_path = file_path
        os.makedirs(os.path.dirname(os.path.abspath(file_path)), exist_ok=True)
        self.df = _load_df(file_path)

    def log_reading(self, data: Dict) -> bool:
        try:
            new_row = pd.DataFrame([{
                'timestamp':    datetime.now(),
                'location_id':  data['location_id'],
                'temp_c':       data['temp_c'],
                'humidity_pct': data['humidity_pct'],
                'wind_kph':     data['wind_kph'],
                'condition':    data['condition'],
                'pressure_hpa': data.get('pressure_hpa', None)
            }])
            self.df = pd.concat([self.df, new_row], ignore_index=True)
            self.df.to_csv(self.file_path, index=False)
            return True
```

```
    except Exception as e:
        logging.error(f"Failed to save: {e}")
        return False


def log_reading_for_date(self, data: Dict, dt: 'datetime') -> bool:
    """Save a forecast row with a specific date (not now)."""
    try:
        # Only write if we do not already have a row for this city+date
        date_str = dt.strftime('%Y-%m-%d')
        location = data['location_id']
        if len(self.df) > 0 and 'location_id' in self.df.columns:
            self.df['_date'] = self.df['timestamp'].dt.strftime('%Y-%m-%d')
            already = self.df[
                (self.df['location_id'] == location) &
                (self.df['_date'] == date_str)
            ]
            self.df.drop(columns=['_date'], inplace=True)
            if len(already) > 0:
                return True   # already have data for this day, skip
        new_row = pd.DataFrame([{
            'timestamp':    dt,
            'location_id':  location,
            'temp_c':       data['temp_c'],
            'humidity_pct': data['humidity_pct'],
            'wind_kph':     data['wind_kph'],
            'condition':    data['condition'],
            'pressure_hpa': data.get('pressure_hpa', None)
```

```
            }])
            self.df = pd.concat([self.df, new_row], ignore_index=True)
            self.df.to_csv(self.file_path, index=False)
            return True
        except Exception as e:
            logging.error(f"Failed to save forecast row: {e}")
            return False

    def get_history(self, city: str = None, days: int = 30) -> List:
        if len(self.df) == 0:
            return []

        df_filtered = self.df.copy()
        if city:
            df_filtered = df_filtered[
                df_filtered['location_id'].str.contains(city, case=False, na=False)
            ]

        cutoff = datetime.now() - timedelta(days=days)
        df_filtered = df_filtered[df_filtered['timestamp'] >= cutoff]
        df_filtered = df_filtered.sort_values('timestamp')

        # Add a date column and keep only the LAST reading per day
        df_filtered['date'] = df_filtered['timestamp'].dt.strftime('%Y-%m-%d')
        df_filtered = df_filtered.drop_duplicates(subset='date', keep='last')

        rows = []
```

```
        for _, row in df_filtered.iterrows():
            ts = row['timestamp']
            if pd.isna(ts):
                continue
            rows.append((
                str(row['date']),
                float(row['temp_c']),
                int(row['humidity_pct']),
                int(row['pressure_hpa']) if pd.notna(row.get('pressure_hpa')) else 1013,
                float(row['wind_kph']),
                str(row['condition'])
            ))
        return rows

    def get_all_cities(self) -> List[str]:
        if len(self.df) == 0:
            return []
        cities = self.df['location_id'].unique()
        city_names = [str(c).split(',')[0].strip() for c in cities if isinstance(c, str)]
        return sorted(list(set(city_names)))


# ====================== WEATHER PROCESSOR ======================
class WeatherProcessor:
    def __init__(self, file_path: str = 'telemetry_logs/daily_telemetry.csv'):
        self.base_url    = "https://geocoding-api.open-meteo.com/v1/search"
        self.weather_url = "https://api.open-meteo.com/v1/forecast"
```

```
    self.file_path   = file_path
    self.df          = _load_df(file_path)   # shared read — uses same fix
    self.storage     = TelemetryStorageEngine(file_path)
    logging.info("✅ WeatherBackend initialized with Open-Meteo (free API)")

def _find_city_coordinates(self, city: str) -> Dict:
    params = {'name': city, 'count': 1, 'language': 'en', 'format': 'json'}
    try:
        response = requests.get(self.base_url, params=params, timeout=10)
        if response.status_code != 200:
            return {'success': False, 'error': "Geocoding service error"}
        data = response.json()
        if not data.get('results'):
            return {'success': False, 'error': f"City '{city}' not found"}
        result = data['results'][0]
        return {
            'success':   True,
            'name':      result['name'],
            'country':   result.get('country', 'Unknown'),
            'latitude':  result['latitude'],
            'longitude': result['longitude']
        }
    except requests.exceptions.ConnectionError:
        return {'success': False, 'error': "Network error - check internet"}
    except Exception as e:
        return {'success': False, 'error': f"Error: {str(e)}"}
```

```
    def description(self, weather_code: int) -> str:
        weather_codes = {
            0: "Clear sky", 1: "Mainly clear", 2: "Partly cloudy", 3: "Overcast",
            45: "Foggy", 48: "Depositing rime fog",
            51: "Light drizzle", 53: "Moderate drizzle", 55: "Dense drizzle",
            56: "Light freezing drizzle", 57: "Dense freezing drizzle",
            61: "Slight rain", 63: "Moderate rain", 65: "Heavy rain",
            66: "Light freezing rain", 67: "Heavy freezing rain",
            71: "Slight snow fall", 73: "Moderate snow fall", 75: "Heavy snow fall",
            77: "Snow grains", 80: "Slight rain showers", 81: "Moderate rain showers",
            82: "Violent rain showers", 85: "Slight snow showers", 86: "Heavy snow
showers",
            95: "Thunderstorm", 96: "Thunderstorm with slight hail",
            99: "Thunderstorm with heavy hail"
        }
        return weather_codes.get(weather_code, "Unknown")


    def _get_weather_icon(self, weather_code: int) -> str:
        if weather_code in [0, 1]:          return "
        elif weather_code in [2, 3]:        return "
        elif weather_code in [45, 48]:      return "
        elif weather_code in [51,53,55,56,57,61,63,65,66,67,80,81,82]: return "
        elif weather_code in [71,73,75,77,85,86]: return "
        elif weather_code in [95,96,99]:    return "
        else:                               return "
```

```
def _save_weather_data(self, weather_data: Dict) -> None:
    telemetry_data = {
        'location_id':  f"{weather_data['city']}, {weather_data['country']}",
        'temp_c':       weather_data['temperature'],
        'humidity_pct': weather_data['humidity'],
        'wind_kph':     weather_data['wind_speed'] * 3.6,
        'condition':    weather_data['condition']
    }
    if weather_data.get('pressure') != 'N/A':
        telemetry_data['pressure_hpa'] = weather_data['pressure']
    success = self.storage.log_reading(telemetry_data)
    if success:
        logging.info(f"📊 Saved weather data for {weather_data['city']}")
    else:
        logging.warning(f"⚠ Failed to save weather data for {weather_data['city']}")

def get_current_weather(self, city: str) -> Dict:
    location = self._find_city_coordinates(city)
    if not location['success']:
        return {'success': False, 'error': location['error']}

    params = {
        'latitude':  location['latitude'],
        'longitude': location['longitude'],
```

```
            'current':
'temperature_2m,relative_humidity_2m,apparent_temperature,weather_code,wind_speed
_10m,pressure_msl',
            'hourly':    'temperature_2m,weather_code',
            'daily':
'weather_code,temperature_2m_max,temperature_2m_min,precipitation_probability_ma
x,wind_speed_10m_max,relative_humidity_2m_mean,pressure_msl_mean',
            'timezone':  'auto'
        }

        try:
            response = requests.get(self.weather_url, params=params, timeout=10)
            response.raise_for_status()
            data    = response.json()
            current = data['current']

            # ── hourly forecast: next 8 hours ──────────────────────────────
            # Use the index into the hourly list, not the raw hour integer.
            # Find which index corresponds to "now" by matching the current time string.
            hourly_times = data['hourly']['time']   # list of "YYYY-MM-DDTHH:00"
            now_str = datetime.now().strftime('%Y-%m-%dT%H:00')

            # Find the closest matching index (fall back to 0 if not found)
            start_idx = 0
            for i, t in enumerate(hourly_times):
                if t >= now_str:
                    start_idx = i
                    break
```

```
        hourly_data = []
        for i in range(8):
            idx = start_idx + i
            if idx < len(hourly_times):
                # Format the time string from the ISO timestamp, not the raw hour int
                t_str   = hourly_times[idx]          # "2024-03-25T09:00"
                t_label = 'Now' if i == 0 else
datetime.fromisoformat(t_str).strftime('%H:%M')
                hourly_data.append({
                    'time': t_label,
                    'icon': self._get_weather_icon(data['hourly']['weather_code'][idx]),
                    'temp': data['hourly']['temperature_2m'][idx]
                })

        # ── 7-day forecast ──────────────────────────────────────────────
        daily_data = []
        for i in range(min(7, len(data['daily']['time']))):
            daily_data.append({
                'day':  datetime.fromisoformat(data['daily']['time'][i]).strftime('%a'),
                'icon': self._get_weather_icon(data['daily']['weather_code'][i]),
                'high': data['daily']['temperature_2m_max'][i],
                'low':  data['daily']['temperature_2m_min'][i],
                'rain': data['daily']['precipitation_probability_max'][i]
            })

        weather_info = {
```

```
    'success':    True,
    'city':       location['name'],
    'country':    location['country'],
    'temp':       current['temperature_2m'],
    'feels_like': current['apparent_temperature'],
    'humidity':   current['relative_humidity_2m'],
    'pressure':   current.get('pressure_msl', 1013),
    'wind':       round(current['wind_speed_10m'] * 3.6, 1),
    'visibility': 10.0,
    'condition':  self.description(current['weather_code']),
    'icon':       self._get_weather_icon(current['weather_code']),
    'uv_index':   5.0,
    'sunrise':    "06:00",
    'sunset':     "18:00",
    'hourly':     hourly_data,
    'forecast':   daily_data
}

# Save today's actual reading
self._save_weather_data({
    'city':       location['name'],
    'country':    location['country'],
    'temperature': current['temperature_2m'],
    'feels_like':  current['apparent_temperature'],
    'humidity':    current['relative_humidity_2m'],
    'pressure':    current.get('pressure_msl', 'N/A'),
    'condition':   self.description(current['weather_code']),
```

```
            'wind_speed':  current['wind_speed_10m']
        })

        # Also save each forecast day so history fills up with real dates
        daily = data['daily']
        for i in range(min(7, len(daily['time']))):
            # skip day 0 — that is today, already saved above
            if i == 0:
                continue
            forecast_dt = datetime.fromisoformat(daily['time'][i])
            # use avg of high+low as the day temperature
            avg_temp = (daily['temperature_2m_max'][i] +
daily['temperature_2m_min'][i]) / 2
            wind_kph  = daily.get('wind_speed_10m_max', [None]*8)[i]
            humidity  = daily.get('relative_humidity_2m_mean', [None]*8)[i]
            pressure  = daily.get('pressure_msl_mean', [None]*8)[i]
            self.storage.log_reading_for_date({
                'location_id':  f"{location['name']}, {location['country']}",
                'temp_c':       avg_temp,
                'humidity_pct': int(humidity) if humidity is not None else 70,
                'wind_kph':     float(wind_kph) if wind_kph is not None else 10.0,
                'condition':    self.description(daily['weather_code'][i]),
                'pressure_hpa': float(pressure) if pressure is not None else 1013.0,
            }, forecast_dt)

        return weather_info
```

```
        except requests.exceptions.ConnectionError:
            return {'success': False, 'error': "Network error - please check your internet connection"}
        except requests.exceptions.Timeout:
            return {'success': False, 'error': "Request timed out - API took too long to respond"}
        except Exception as e:
            return {'success': False, 'error': f"Unexpected error: {str(e)}"}

    def compare_cities(self, city1: str, city2: str) -> Dict:
        weather1 = self.get_current_weather(city1)
        weather2 = self.get_current_weather(city2)
        if not weather1['success']:
            return {'success': False, 'error': f"Couldn't get data for {city1}: {weather1['error']}"}
        if not weather2['success']:
            return {'success': False, 'error': f"Couldn't get data for {city2}: {weather2['error']}"}
        return {'success': True, 'city1': weather1, 'city2': weather2}

    def get_history(self, city: str, days: int = 30) -> List:
        return self.storage.get_history(city, days)

    def get_all_cities(self) -> List[str]:
        return self.storage.get_all_cities()

    def find_extremes(self):
        if len(self.df) == 0:
```

```
        return {'success': False, 'error': 'No data available'}
    h_idx   = self.df['temp_c'].idxmax()
    c_idx   = self.df['temp_c'].idxmin()
    hottest = self.df.iloc[h_idx]
    coldest = self.df.iloc[c_idx]
    return {
        'success': True,
        'hottest': {'temperature': hottest['temp_c'], 'city': hottest['location_id'], 'timestamp': str(hottest['timestamp'])},
        'coldest': {'temperature': coldest['temp_c'], 'city': coldest['location_id'], 'timestamp': str(coldest['timestamp'])}
    }


def calculate_trends(self, limit: int = 5):
    if len(self.df) < 2:
        return {'success': False, 'error': 'Not enough data for trend analysis'}
    self.df['temp_slope']     = self.df['temp_c'].diff()
    self.df['smooth_signal']  = self.df['temp_c'].rolling(window=min(3, len(self.df))).mean()
    last_entries = self.df.tail(limit)
    trends = []
    for _, row in last_entries.iterrows():
        trends.append({
            'timestamp':    str(row['timestamp']),
            'temperature':  float(row['temp_c']),
            'slope':        float(row['temp_slope']) if not pd.isna(row['temp_slope']) else 0,
            'smooth_signal':float(row['smooth_signal']) if not pd.isna(row['smooth_signal']) else float(row['temp_c'])
```

```
            })
        if len(trends) >= 2:
            overall_change = trends[-1]['temperature'] - trends[0]['temperature']
            overall_trend  = 'increasing' if overall_change > 0 else ('decreasing' if
overall_change < 0 else 'stable')
        else:
            overall_trend  = 'insufficient_data'
            overall_change = 0
        return {
            'success':           True,
            'trends':            trends,
            'overall_trend':     overall_trend,
            'temperature_change': overall_change,
            'total_records':     len(self.df),
            'last_updated':      str(self.df['timestamp'].max()) if len(self.df) > 0 else None
        }


def get_current_weather(city: str) -> Dict:
    return WeatherProcessor().get_current_weather(city)


if __name__ == "__main__":
    processor = WeatherProcessor()
    print("Testing weather backend...")
    result = processor.get_current_weather("London")
    if result['success']:
        print(f"Got weather for {result['city']}: {result['temp']}C")
        print(f"Hourly sample: {result['hourly'][:3]}")
```

```
        print(f"Forecast sample: {result['forecast'][:2]}")
    else:
        print(f"Error: {result['error']}")
```

Code Linking GUI & Backend

```
import sys
import os
from PySide6.QtCore import QThread, Signal, Qt
from PySide6.QtWidgets import QApplication, QPushButton
import finalBackend
from design_ai import MainWindow, SplashScreen

sys.stdout = os.fdopen(sys.stdout.fileno(), 'w', buffering=1)

# Make all file paths relative to THIS script's directory, not cwd
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
print("=" * 60)
print("WEATHER APP BRIDGE")
print("=" * 60)

class WeatherFetcher(QThread):
    success = Signal(dict)
    failure = Signal(str)

    def __init__(self, city):
```

```
        super().__init__()
        self.city = city

    def run(self):
        try:
            print(f"Fetching {self.city}...")
            csv_path = os.path.join(SCRIPT_DIR, 'test_logs', 'daily_telemetry.csv')
            processor = finalBackend.WeatherProcessor(file_path=csv_path)
            result = processor.get_current_weather(self.city)
            if result.get('success'):
                print(f"Got {result['city']}: {result['temp']}C")
                self.success.emit(result)
            else:
                error = result.get('error', 'Unknown error')
                print(f"Error: {error}")
                self.failure.emit(error)
        except Exception as e:
            print(f"Exception: {e}")
            import traceback; traceback.print_exc()
            self.failure.emit(str(e))

def _make_processor():
    """Always use the same absolute CSV path regardless of cwd."""
    csv_path = os.path.join(SCRIPT_DIR, 'test_logs', 'daily_telemetry.csv')
    return finalBackend.WeatherProcessor(file_path=csv_path)

def main():
```

```
print("Starting application...")
app = QApplication(sys.argv)
app.setStyle("Fusion")

splash = SplashScreen()
splash.show()

_threads = []
def _cleanup(thread):
    thread.quit()
    thread.wait(200)
    if thread in _threads:
        _threads.remove(thread)

def _update_city_lists(w):
    """Reload cities from disk and repopulate both dropdowns."""
    try:
        processor = _make_processor()
        cities = processor.get_all_cities()
        print(f"Cities in dropdown: {cities}")
        w.th.set_cities(cities)
        w.tt.set_cities(cities)
    except Exception as e:
        print(f"City list error: {e}")

def launch_main():
    splash.close()
```

```
w = MainWindow()
w.showMaximized()

# ── CURRENT TAB ──────────────────────────────────────────────
def on_current_search():
    city = w.tc.search.text().strip()
    if not city:
        return
    print(f"Search: '{city}'")
    w.tc.show_loading()
    t = WeatherFetcher(city)
    _threads.append(t)

    def on_ok(data):
        w.tc.load_data(data)
        _update_city_lists(w)   # refresh dropdowns after every successful search
        _cleanup(t)

    def on_err(msg):
        w.tc.show_error(msg)
        _cleanup(t)

    t.success.connect(on_ok, Qt.ConnectionType.QueuedConnection)
    t.failure.connect(on_err, Qt.ConnectionType.QueuedConnection)
    t.start()
```

```
try:
    w.tc.search.returnPressed.disconnect()
except RuntimeError:
    pass
w.tc.search.returnPressed.connect(on_current_search)

for btn in w.tc.findChildren(QPushButton):
    if btn.text() == "Search":
        try:
            btn.clicked.disconnect()
        except RuntimeError:
            pass
        btn.clicked.connect(on_current_search)
        break
w.tc._fetch = on_current_search

# ── COMPARE TAB ──────────────────────────────────────────────
def on_compare():
    city1 = w.tcm.c1.text().strip()
    city2 = w.tcm.c2.text().strip()
    if not city1 or not city2:
        w.tcm.show_error("Enter both cities.")
        return
    print(f"Compare: '{city1}' vs '{city2}'")
    w.tcm.show_loading()
    w.tcm._d1 = None
    w.tcm._d2 = None
```

```
t1 = WeatherFetcher(city1)
t2 = WeatherFetcher(city2)
_threads.extend([t1, t2])

def ok1(data):
    w.tcm.load_city1(data['city'], data)
    _cleanup(t1)

def err1(msg):
    w.tcm.show_error(f"{city1}: {msg}")
    _cleanup(t1)

def ok2(data):
    w.tcm.load_city2(data['city'], data)
    _cleanup(t2)

def err2(msg):
    w.tcm.show_error(f"{city2}: {msg}")
    _cleanup(t2)

t1.success.connect(ok1, Qt.ConnectionType.QueuedConnection)
t1.failure.connect(err1, Qt.ConnectionType.QueuedConnection)
t2.success.connect(ok2, Qt.ConnectionType.QueuedConnection)
t2.failure.connect(err2, Qt.ConnectionType.QueuedConnection)
t1.start()
t2.start()
```

```
for btn in w.tcm.findChildren(QPushButton):
    if btn.text() == "Compare":
        try:
            btn.clicked.disconnect()
        except RuntimeError:
            pass
        btn.clicked.connect(on_compare)
        break
w.tcm._compare = on_compare

# ── HISTORY TAB ──────────────────────────────────────────────
def on_load_history():
    city = w.th.combo.currentText()
    print(f"Load history clicked, city='{city}'")
    if not city:
        print("No city selected in dropdown")
        return
    try:
        processor = _make_processor()
        rows = processor.get_history(city, days=30)
        print(f"Found {len(rows)} history records for {city}")
        w.th.load_history(rows)
    except Exception as e:
        print(f"History error: {e}")
        import traceback; traceback.print_exc()
```

```
# Disconnect whatever the frontend wired up and connect our real handler
for btn in w.th.findChildren(QPushButton):
    if btn.text() == "Load":
        try:
            btn.clicked.disconnect()
        except RuntimeError:
            pass
        btn.clicked.connect(on_load_history)
        print("Connected Load button")
        break
w.th._load = on_load_history

# ── TRENDS TAB ──────────────────────────────────────────────
def on_plot():
    city = w.tt.cc.currentText()
    print(f"Plot clicked, city='{city}'")
    if not city:
        print("No city selected in trends dropdown")
        return
    try:
        processor = _make_processor()
        rows = processor.get_history(city, days=30)
        print(f"Found {len(rows)} trend records for {city}")
        w.tt.load_trends(rows)
    except Exception as e:
        print(f"Trends error: {e}")
        import traceback; traceback.print_exc()
```

```
for btn in w.tt.findChildren(QPushButton):
    if btn.text() == "Plot":
        try:
            btn.clicked.disconnect()
        except RuntimeError:
            pass
        btn.clicked.connect(on_plot)
        print("Connected Plot button")
        break
w.tt._plot = on_plot

# ——— Refresh dropdowns when the user switches to History or Trends tab ———
# This ensures cities searched earlier in the session always appear.
tab_widget = w.centralWidget().findChild(
    __import__('PySide6.QtWidgets', fromlist=['QTabWidget']).QTabWidget
)
if tab_widget:
    def on_tab_changed(index):
        # Tab 2 = History, Tab 3 = Trends
        if index in (2, 3):
            _update_city_lists(w)
    tab_widget.currentChanged.connect(on_tab_changed)

# Populate on startup (catches cities from previous sessions)
_update_city_lists(w)
```

```
    try:
        splash.btn.clicked.disconnect()
    except RuntimeError:
        pass
    splash.btn.clicked.connect(launch_main)

    print("App ready! Click Get Started.")
    sys.exit(app.exec())

if __name__ == "__main__":
    main()
```

To run the program

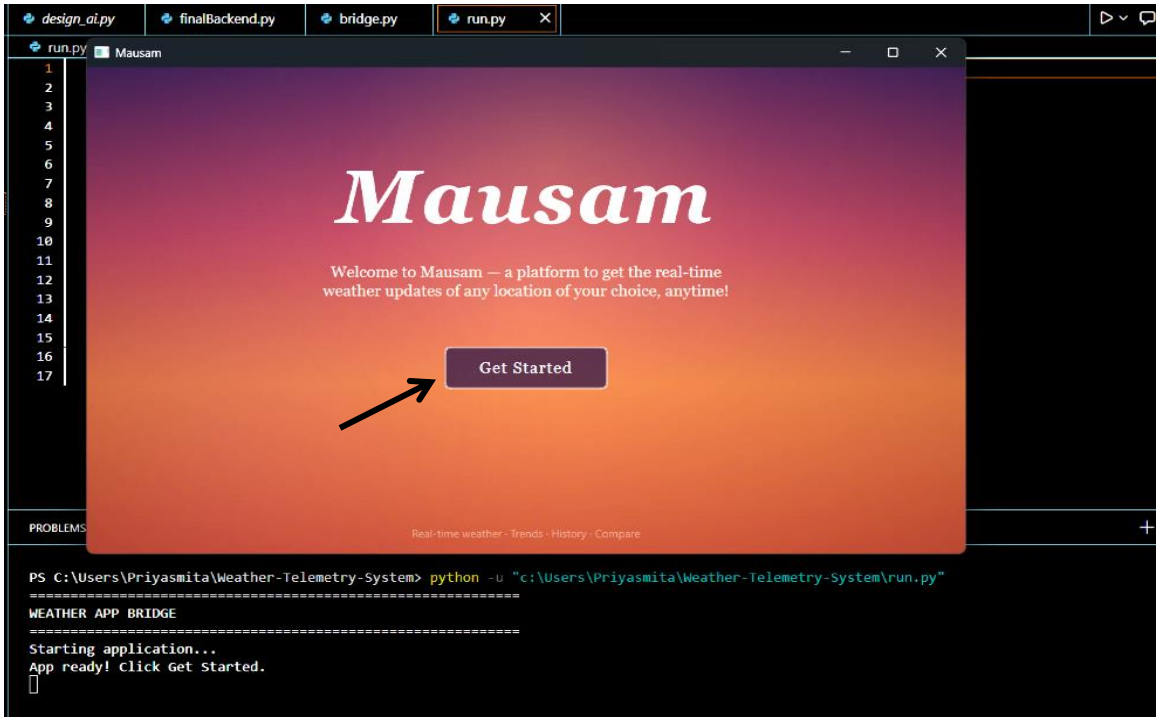
```
# run_weather_app.py
"""
Launcher for the Weather App
Run this file to start the application
"""

import sys
import os

# Add current directory to path
sys.path.insert(0, os.path.dirname(__file__))
```

```
# Import and run the controller
from bridge import main
if __name__ == "__main__":
    main()
```

Output:

Step 1 – From the Git repository "**Project-Mausam**", pull the repository into VS Code. Then, go to all-files branch, and run the run.py file. (Make sure to download all the required packages listed previously to run this file)

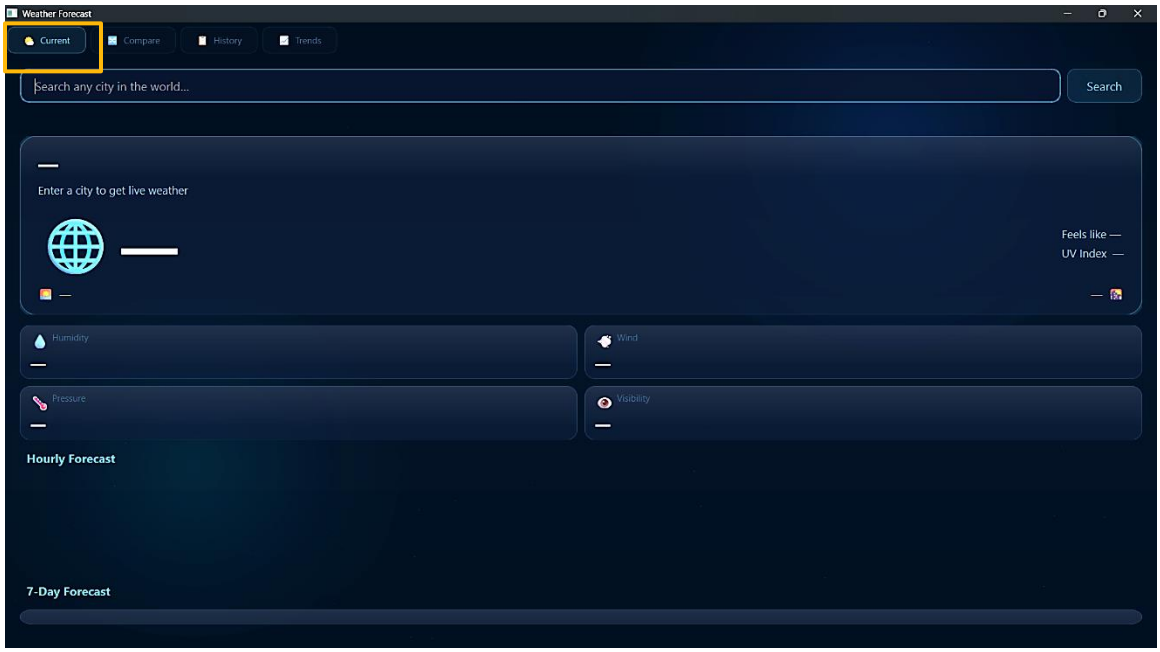
It'll lead the user to the below widget ⬇



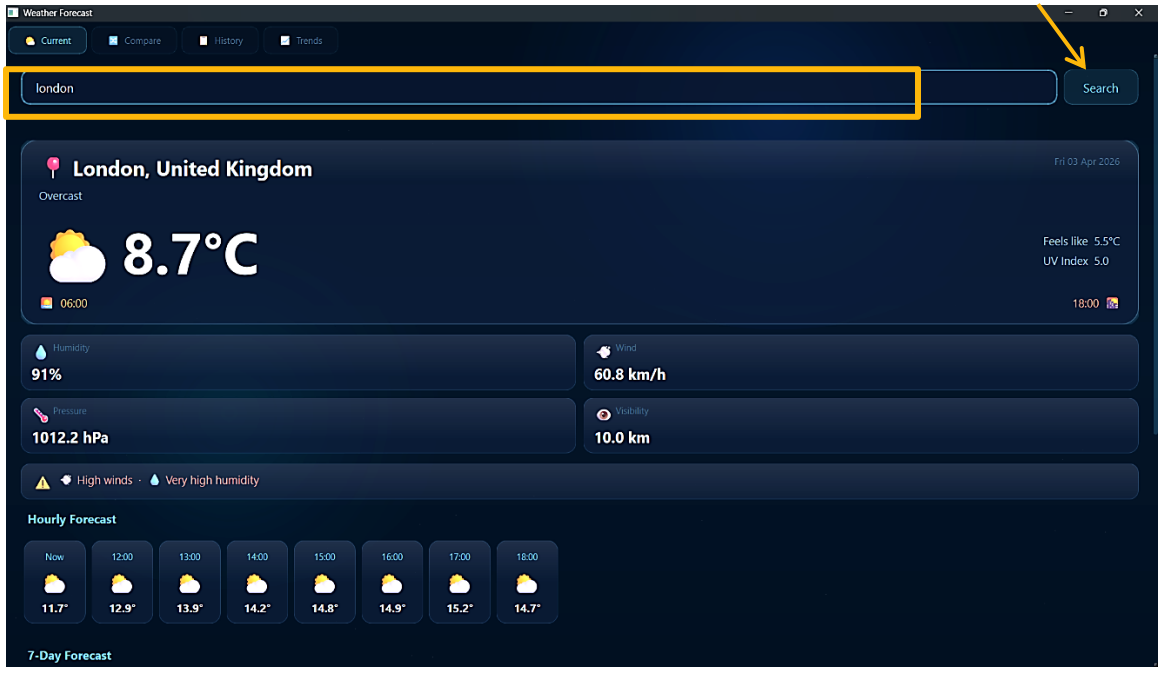
Click on the "**Get Started**" button.

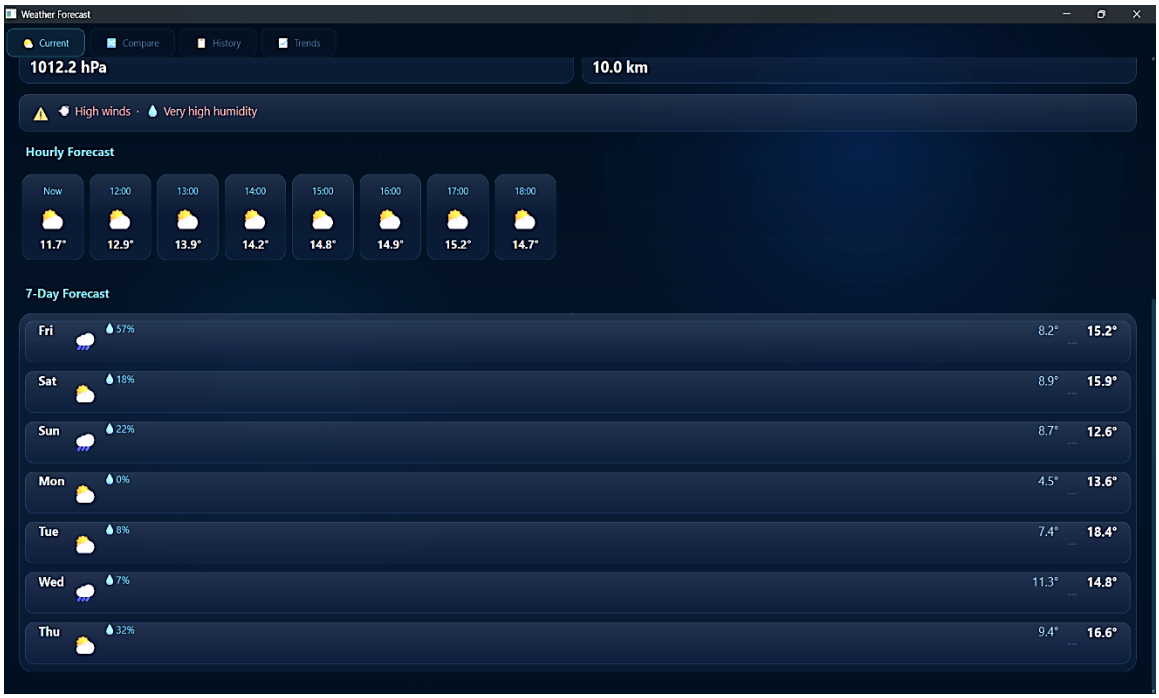
Step 2 – After clicking the button, it'll redirect the user to the following tab ⬇

This is the "**Current Tab**". Here, we get temperature, humidity, UV Index, Wind, etc. about any particular city.



Step 3 – Now, the user can type any city name in the search box and click **Search** to get its weather report along with next 8-hours Forecast and 7-day Forecast.





<u>Step 4</u> – Moving on to the next tab, "**Compare Tab**". Here, the user can enter any 2 cities to compare their weather. Click on the **Compare** button to get detailed comparison in the form Bar graph.

<u>Step 5</u> – In the next tab "**History Tab**", the user can see the hottest & the coldest day in a 7-day period along with History of upcoming 7-days of a city in the drop-down menu, by clicking **Load** button. User can also export this data in an excel file using the "**Export to CSV**" button at the bottom right of the tab.

<u>Step 6</u> – In the last tab "**Trends Tab**", the user will be able to see trends of temperature, humidity, wind speed & pressure in the form of line graph when the user enters a city from the drop-down menu & a particular metric then click **Plot** button, as shown below⬇

Google Drive link to the explanation video:
https://drive.google.com/file/d/1Wus_Tj24MLHe3UqMSaZwhOUMl_VrA4B7/view?usp=drivesdk